TUTORIAL DE GIT

Capítulo 1. Inicio: sobre el control de versiones

Juan Felipe Castañeda Rodríguez · Fundación Universitaria Agraria de Colombia (UNIAGRARIA)

Ciencia de Datos — Actividad 2: Hands on, configuración de herramientas · Octubre de 2026

Este primer capítulo explica qué es el control de versiones, por qué existe Git, cómo instalarlo y cómo dejarlo configurado antes de empezar a trabajar. Es la base de todo lo que viene en el resto del tutorial.

1.1 ¿Qué es un sistema de control de versiones?

Un sistema de control de versiones (VCS, por sus siglas en inglés) es una herramienta que registra los cambios hechos sobre un conjunto de archivos a lo largo del tiempo, de modo que se pueda volver a cualquier estado anterior. Aunque se asocia con código fuente, sirve para cualquier tipo de archivo: informes, planos, configuraciones o datos.

Con un VCS es posible:

- Revertir un archivo o un proyecto completo a una versión anterior.
- Comparar cambios entre versiones y saber quién modificó qué y cuándo.
- Recuperar archivos borrados o dañados por error.
- Trabajar en paralelo con otras personas sin sobrescribir el trabajo ajeno.

1.2 Tipos de sistemas de control de versiones

Sistemas locales. El método más simple es copiar carpetas con nombres como "proyecto_v2_final". Herramientas como RCS automatizaron esto guardando las diferencias entre versiones en el propio disco, pero solo sirven para una persona en una máquina.

Sistemas centralizados (CVCS). Herramientas como Subversion o CVS usan un único servidor que guarda todas las versiones; cada persona descarga una copia de trabajo. Facilitan la colaboración, pero el servidor es un punto único de falla: si se cae, nadie puede guardar cambios, y si se daña su disco sin respaldo, se pierde la historia.

Sistemas distribuidos (DVCS). En Git, Mercurial o Bazaar cada persona clona el repositorio completo, incluida toda su historia. Cada clon es un respaldo íntegro, se puede trabajar sin conexión y se habilitan flujos de trabajo mucho más flexibles.

1.3 Breve historia de Git

Git nació en 2005 en la comunidad del kernel de Linux. Hasta ese momento el proyecto usaba BitKeeper, un DVCS propietario; cuando se retiró la licencia gratuita, Linus Torvalds diseñó una herramienta propia con estos objetivos: velocidad, diseño simple, soporte fuerte para desarrollo no lineal (miles de ramas en paralelo), arquitectura totalmente distribuida y capacidad para manejar proyectos enormes de forma eficiente.

1.4 Fundamentos: cómo piensa Git

Instantáneas, no diferencias. Muchos sistemas guardan la lista de cambios de cada archivo. Git, en cambio, guarda una instantánea (snapshot) del estado de todos los archivos en cada confirmación. Si un archivo no cambió, solo guarda un enlace al archivo idéntico anterior.

Casi todo es local. Como la historia completa está en el disco, consultar el historial o comparar versiones es prácticamente instantáneo y no requiere red.

Integridad. Todo se identifica con una suma de comprobación SHA-1 (una cadena de 40 caracteres hexadecimales, por ejemplo 24b9da6552252987aa493b52f8696cd6d3b00373). Es imposible modificar un archivo sin que Git lo detecte.

Git generalmente solo añade datos. Casi todas las acciones agregan información a la base de datos; una vez confirmado algo, es muy difícil perderlo.

Los tres estados. Un archivo en Git puede estar:

- **Modificado (modified):** cambió, pero aún no se ha marcado para guardar.
- **Preparado (staged):** se marcó para ir en la próxima confirmación.
- **Confirmado (committed):** quedó guardado en la base de datos local.

Esto da lugar a las tres secciones de un proyecto: el directorio de trabajo (los archivos que editas), el área de preparación o staging area (lo que irá en el próximo commit) y el directorio .git (la base de datos del repositorio).

El flujo básico es: modificar archivos, preparar los cambios con git add y confirmarlos con git commit.

1.5 La línea de comandos

Existen muchas interfaces gráficas para Git, pero la línea de comandos es el único lugar donde están disponibles todos los comandos. Por eso este tutorial usa la terminal (Git Bash en Windows, Terminal en macOS o Linux).

1.6 Instalación de Git

Windows: descargar el instalador desde https://git-scm.com/download/win y seguir el asistente. Se instala también Git Bash, una terminal con comandos tipo Unix.

macOS: instalar las herramientas de línea de comandos de Xcode o usar Homebrew:

```
brew install git
```

Linux (Debian/Ubuntu):

```
sudo apt install git-all
```

Para comprobar la instalación:

```
git --version
```

1.7 Configuración inicial

Git usa el comando git config para leer y escribir variables de configuración en tres niveles: sistema (--system), usuario (--global) y repositorio (--local). Lo primero es indicar la identidad, porque cada commit la incluye:

```
git config --global user.name "Tu Nombre"
git config --global user.email "tu_correo@ejemplo.com"
```

Opcionalmente se puede elegir el editor de texto y el nombre de la rama por defecto:

```
git config --global core.editor "code --wait"
git config --global init.defaultBranch main
```

Para revisar la configuración:

```
git config --list
git config user.name
```

1.8 Cómo obtener ayuda

Cualquiera de estas formas abre la ayuda de un comando:

```
git help config
git config --help
git add -h
```

La opción -h muestra un resumen corto de las opciones más usadas.

1.9 Resumen

Un sistema de control de versiones guarda la historia de un proyecto. Git es distribuido, guarda instantáneas, trabaja casi todo en local y garantiza la integridad con SHA-1. Sus archivos pasan por tres estados (modificado, preparado y confirmado). Antes de usarlo hay que instalarlo y configurar el nombre y el correo del usuario.

Referencia

Chacon, S., & Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. https://git-scm.com/book/es/v2